*NAME: Krish Kumar Singh*
*E-MAIL: krishkumarsinghpw@gmail.com*
*GitHub:Krish260653*
*LinkedIn: www.linkedin.com/in/krish-kumar-singh-24a161387*

# *TITLE: Enhancing Web Components Support in Calico through DSL Improvements and Code Generation*

## *ABSTRACT*

*Thisproject aims to enhance support for Web Components in Calico, a reactive UI library built on functional programming principles using Cats Effect and FS2. While Web Components provide a standardized and framework-agnostic approach to building reusable UI elements, their integration within Calico is currently limited and lacks ergonomic, type-safe abstractions.*
*The proposed work focuses on extending Calico's domain-specific language (DSL) to provide intuitive and consistent support for Web Components, along with introducing code-generation techniques to automatically create type-safe bindings. This will enable developers to seamlessly integrate existing Web Components while preserving the functional and reactive design principles of Calico.*
*Additionally, the project will address challenges related to lifecycle management, event handling, and interoperability with Web APIs such as Custom Elements and Shadow DOM. The outcome will be a more expressive and developer-friendly system that reduces boilerplate, improves maintainability, and expands Calico's usability within modern web development ecosystems.*

## *PROBLEM*

*Calicocurrently provides limited and low-level support for integrating Web Components, lacking ergonomic and type-safe abstractions. Developers must manually interact with Web APIs to handle attributes, events, and lifecycle behavior, leading to increased boilerplate and potential inconsistencies.*
*Additionally, the absence of automated, type-safe bindings makes it difficult to scale the use of different Web Components within Calico applications. Integrating these dynamic components with Calico's reactive and functional model also introduces challenges in maintaining correctness and consistency.*
*These limitations reduce developer productivity and hinder the seamless adoption of Web Components in Calico-based applications.*

## *SOLUTION*

*Thisproject proposes to enhance Web Components support in Calico by improving its DSL and introducing a structured approach for generating type-safe bindings. The goal is to make the integration of Web Components more natural, consistent, and aligned with Calico's functional programming model.*

*The first part of the work will focus on extending the existing DSL to provide clearer and more expressive abstractions for working with Web Components. This includes better support for defining attributes, properties, and event handlers in a way that feels consistent with how UI elements are currently expressed in Calico. The intention is to reduce the need for low-level interaction with Web APIs and make component usage more intuitive.*

*The second part involves introducing a code-generation mechanism to automate the creation of Scala bindings for Web Components. Instead of manually writing wrappers, this approach will allow developers to generate type-safe interfaces, ensuring correctness while significantly reducing boilerplate. This will also make it easier to support a wider range of existing Web Components*

*.*

*In addition, the project will address integration challenges related to lifecycle management and state updates. Web Components will be connected more effectively with Calico's reactive model, ensuring that updates and events behave predictably within the system. Special attention will be given to maintaining compatibility with Web standards such as Custom Elements and Shadow DOM.*

*Overall, the solution will be designed to fit naturally within Calico's architecture, keeping the implementation modular and easy to extend. This will not only improve the current developer experience but also provide a strong foundation for future enhancements.*

# System Architecture

*Theproposed architecture extendsCalico's reactive design to support seamless integration of Web Components while maintaining type safety and functional consistency. The system is structured in a modular pipeline that separates abstraction, generation, and runtime concerns.*

## → Processing Flow

*Web Components → DSL Layer → Code Generation → Calico Runtime → UI Rendering*

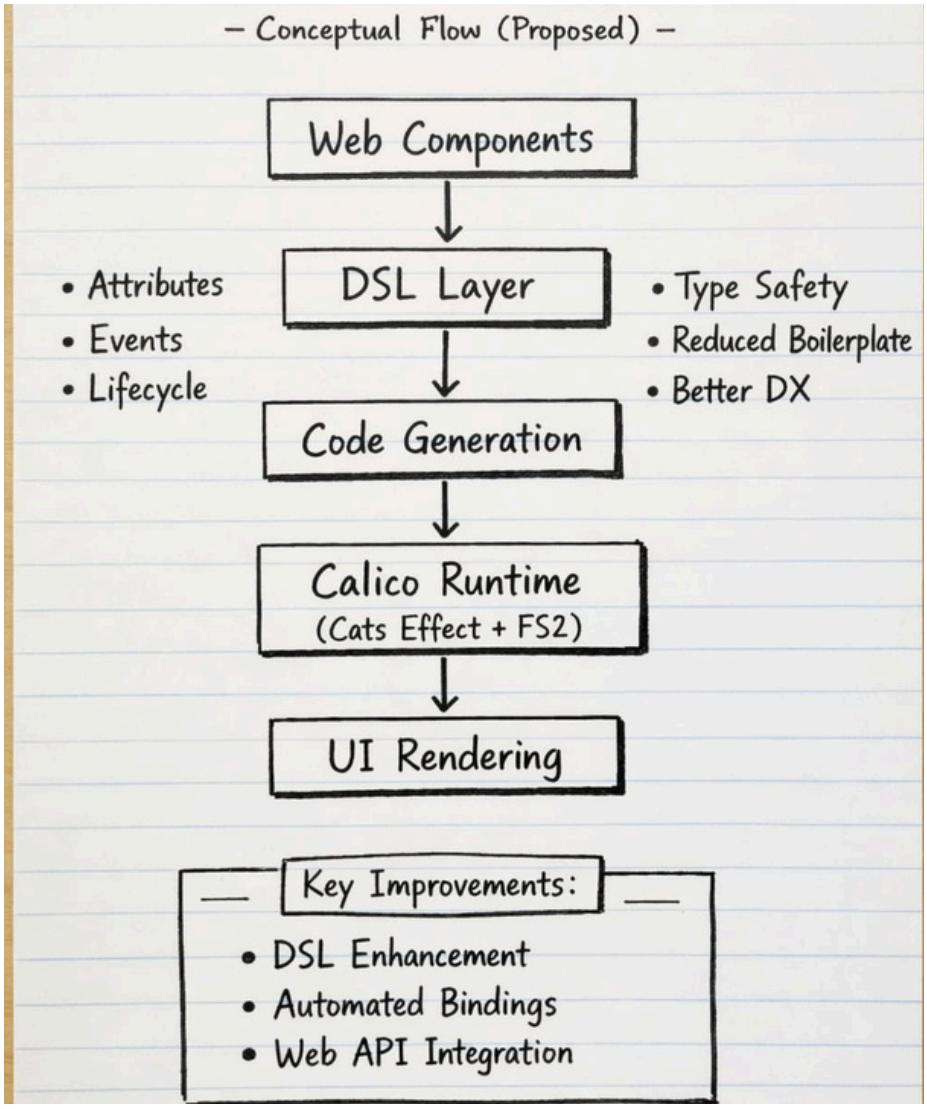
↓ *Reusable UI elements built using browser standards like Custom*

↓ *Provides high-level abstractions to interact with Web Components in a declarative and type-safe way.*

↓ *Automatically generates Scala bindings, reducing manual boilerplate and ensuring consistency.*

↓ *Handles state, effects, and event propagation using Cats Effect and FS2.*

↓ *Responsible for rendering components and synchronizing them with application state.*



## *Deliverables*

*1. Enhanced DSL in Calico for intuitive and type-safe integration of Web Components.*
*2. Code-generation mechanism to automatically create Scala bindings for Web Components.*
*3. Support for handling attributes, properties, and events in a consistent manner.*
*4. Integration with Calico runtime ensuring proper lifecycle and reactive behavior.*
*5. Test cases validating correctness and compatibility with existing components.*
*6. Documentation explaining usage, design decisions, and examples.*

## *Impact*

*1. Enables seamless use of Web Components in Calico, improving interoperability with modern web standards.*
*2. Reduces boilerplate through DSL improvements and automated bindings, increasing developer productivity.*
*3. Improves type safety and consistency when working with dynamic UI components.*
*4. Strengthens Calico's usability for real-world frontend development use cases.*
*5. Contributes reusable infrastructure for future Web Component integrations.*

# Risks and Mitigation

1. Complexity of Web APIs (Custom Elements, Shadow DOM)
   → Start with basic components and incrementally support advanced features

2. Type Safety Challenges in Dynamic Components
   → Use well-defined abstractions and strong typing with thorough testing

3. Integration with Calico Runtime
   → Follow modular design and validate changes with small, incremental updates

4. Code Generation Edge Cases
   → Begin with limited components and generalize gradually

5. Time Management for 350h Scope
   → Prioritize core features (DSL + basic bindings) before advanced optimizations

# Tech Stack

1. Scala
2. Cats Effect
3. FS2
4. Web APIs (Custom Elements,Shadow DOM)
5. SBT, GIT

# *PROJECT TIME-LINE*

## *Community Bonding Period:*

*StudyCalico architecture, understand existing DSL limitations, explore Web Components APIs (Custom Elements, Shadow DOM), and finalize the design approach with the mentor.*

*Weeks 1–3:*
*Analyze current DSL limitations and design improved abstractions for integrating Web Components.*
*Weeks 4–7:*
*Implement DSL enhancements with support for attributes, properties, and event handling.*
*Weeks 8–11:*
*Develop a code-generation mechanism to create type-safe Scala bindings for Web Components and validate with sample components.*
*Weeks 12–14:*
*Integrate generated bindings with Calico runtime and handle lifecycle, state updates, and reactive behavior.*
*Weeks 15–17:*
*Perform testing, fix bugs, benchmark performance, and complete documentation and final polishing.*

# *Why Me?*

*Ihave a strong interest in building reliable and maintainable systems, and this project aligns well with my focus on writing clean and efficient code. I am comfortable working with modern programming concepts and have experience in understanding existing codebases, which will help me quickly get familiar with Calico's architecture.*

*I am particularly interested in functional programming and how it can be applied to real-world problems such as UI development. This project gives me an opportunity to work with technologies like Scala, Cats Effect, and FS2 while solving a practical problem related to Web Components integration.*

*I follow a disciplined development approach, including writing clear code, testing features properly, and maintaining consistency across implementations. I am also proactive in communication and open to feedback, which will help me collaborate effectively with mentors throughout the program.*

*I am committed to completing the project with high quality and would like to continue contributing to the ecosystem beyond the GSoC period.*